

AI Chatbot — System Architecture

A full-stack, streaming AI chatbot: a React (Vite) frontend, a Node.js / Express backend, and the OpenAI API — built and deployed end-to-end, from local development to a live public URL.

React + Vite

Node.js + Express

OpenAI API (gpt-4o-mini)

Vercel · frontend

Render · backend

localStorage persistence

[Try it live](#)

[View source](#)

What it does

A chat interface where replies stream in token-by-token, like ChatGPT, instead of appearing all at once. Built with the same separation real production AI products use: the browser never talks to OpenAI directly.

Streaming replies

Responses appear word-by-word as the model generates them.

Multiple conversations

A sidebar to create, switch between, and delete separate chats.

Persistence

Conversations and theme are saved to the browser, surviving a refresh.

Markdown rendering

Bold text, lists, links, and code blocks render properly, not as raw text.

Dark / light mode

A toggle in the sidebar, remembered between visits.

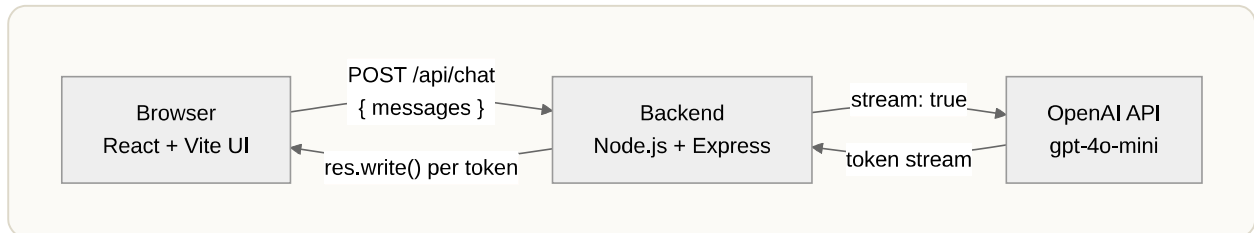
Responsive layout

A collapsible sidebar and mobile-safe viewport handling on phones.

Architecture

Two independent apps talking over HTTP. The backend is the only piece that holds the OpenAI API key — the frontend is a plain static site with no secrets in it.

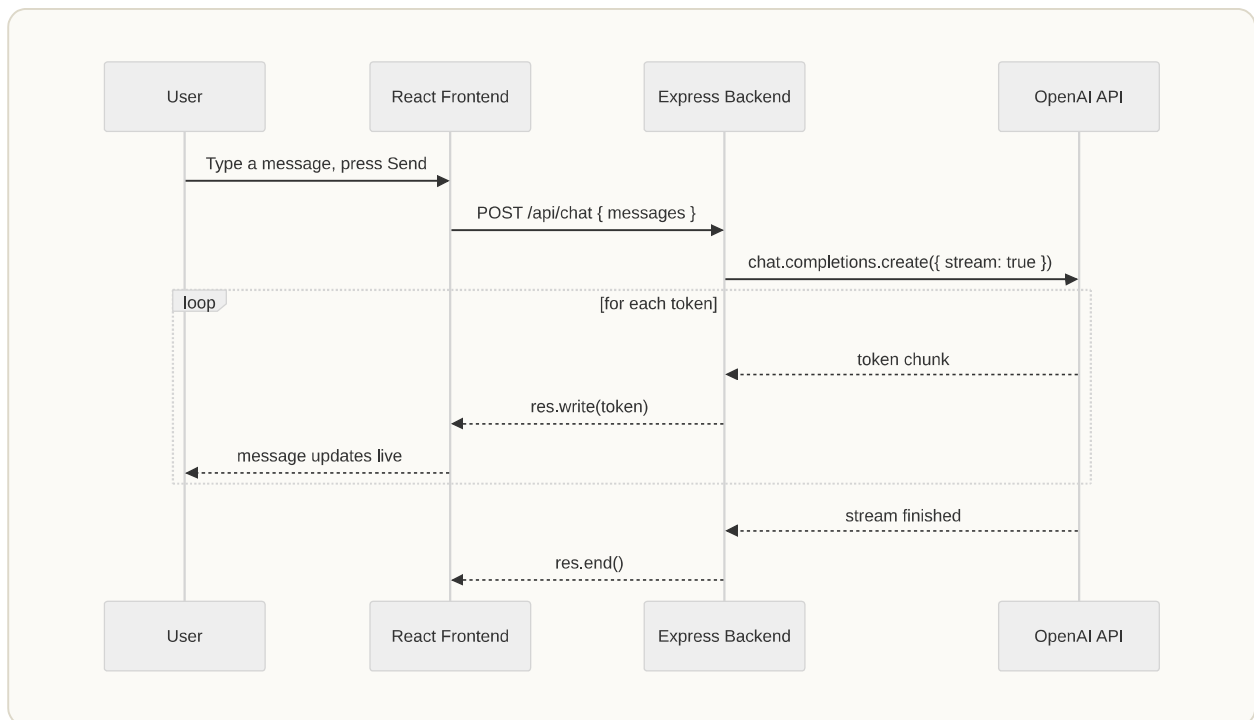
FIG. 01 — SYSTEM ARCHITECTURE



The frontend sends the full conversation to the backend on every message. The backend attaches the API key server-side, calls OpenAI, and pipes the response straight back — the key never reaches the browser.

Request flow

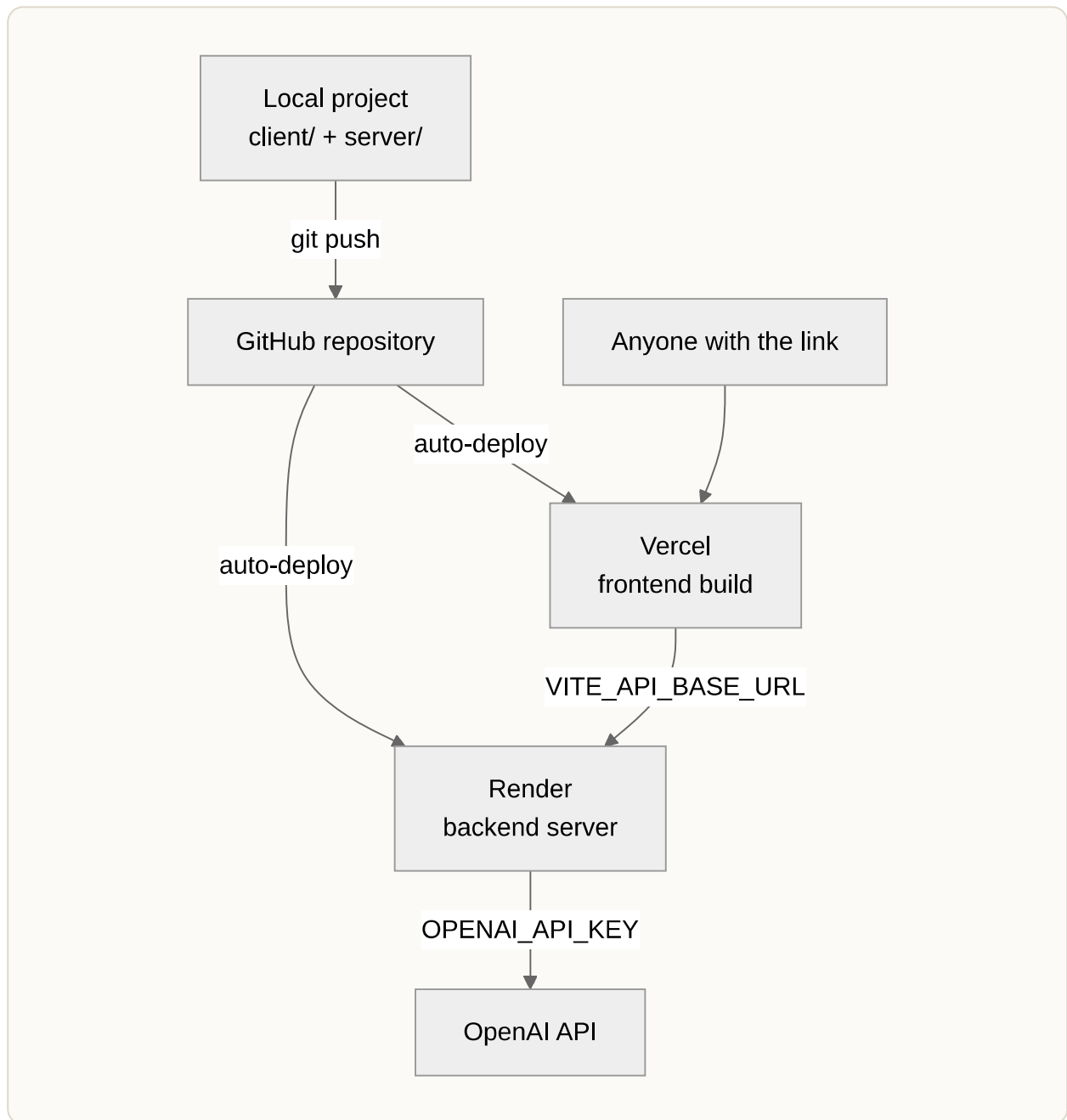
FIG. 02 — A SINGLE MESSAGE, END TO END



Because the reply is streamed rather than awaited as one block, the first word can appear in well under a second, and the UI updates continuously instead of showing a single loading spinner.

Deployment

FIG. 03 — FROM A PUSH TO A LIVE URL



A push to GitHub's main branch redeploys both services automatically — Vercel rebuilds the frontend, Render restarts the backend — so the live site always matches the latest commit.

Notable engineering decisions

Split client / server	Keeps the OpenAI API key off the browser entirely — a security requirement for any real AI product, not just a nicety.
HTTP streaming	Uses <code>res.write()</code> on the backend and a <code>ReadableStream</code> reader on the frontend, so tokens render as they arrive rather than after the full reply completes.
Mobile viewport handling	Android Chrome resizes its visible viewport as the address bar shows and hides; CSS <code>dvh</code> alone isn't fully reliable across devices, so the actual height is also measured in JS and kept in sync.
Local persistence	Conversations and theme are saved to <code>localStorage</code> with defensive fallbacks, so a refresh never loses a chat.

Project structure

```
ai-chatbot-react/
├── client/                # React app (Vite) – the chat UI
│   ├── src/
│   │   ├── App.jsx        # conversations, theme, streaming logic
│   │   ├── components/
│   │   │   ├── Sidebar.jsx
│   │   │   ├── ChatWindow.jsx
│   │   │   ├── Message.jsx
│   │   │   └── TypingIndicator.jsx
│   │   └── lib/storage.js # localStorage helpers
└── server/                # Express app – proxies to OpenAI
    └── server.js
```